



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №7

із дисципліни «Технології розроблення програмного забезпечення»

Тема: «Патерни проектування»

Тема проекту: «Музичний програвач»

GitHub: <https://github.com/martyshenkoo/MusicPlayerWeb>

Виконала:

студентка групи ІА-31:

Мартишенко І.С.

Перевірів:

Мягкий М.Ю.

Мета: Вивчити структуру шаблонів «Mediator», «Facade», «Bridge», «Template method» та навчитися застосовувати їх в реалізації програмної системи.

Музичний програвач (iterator, command, memento, facade, visitor, client-server)

Музичний програвач становить собою програму для програвання музичних файлів або відтворення потокової музики з можливістю створення, запам'ятовування і редагування списків програвання, перемішування/повторення (shuffle/repeat), розпізнавання різних аудіо-форматів, еквалайзер.

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Теоретичні відомості

Призначення патерну: Шаблон «Facade» (фасад) передбачає створення єдиного уніфікованого способу доступу до підсистеми без розкриття внутрішніх деталей підсистеми. Оскільки підсистема може складатися з безлічі класів, а кількість її функцій – не більше десяти, то щоб уникнути створення «спагеті-коду» (коли все тісно пов'язано між собою) виділяють один загальний інтерфейс доступу, здатний правильним чином звертатися до внутрішніх деталей.

Це також відволікає користувачів від змін в підсистемі (внутрішня реалізація може змінюватися, а наданої послуги немає), що також скоротить кількість змін в використовуваних фасад класах (без фасаду довелося б змінювати вихідні коди в безлічі точок). Звичайно, твердої умови повного закриття внутрішніх класів підсистеми не стоїть – при необхідності можна звертатися до окремих класів безпосередньо, минаючи об'єкт фасад.

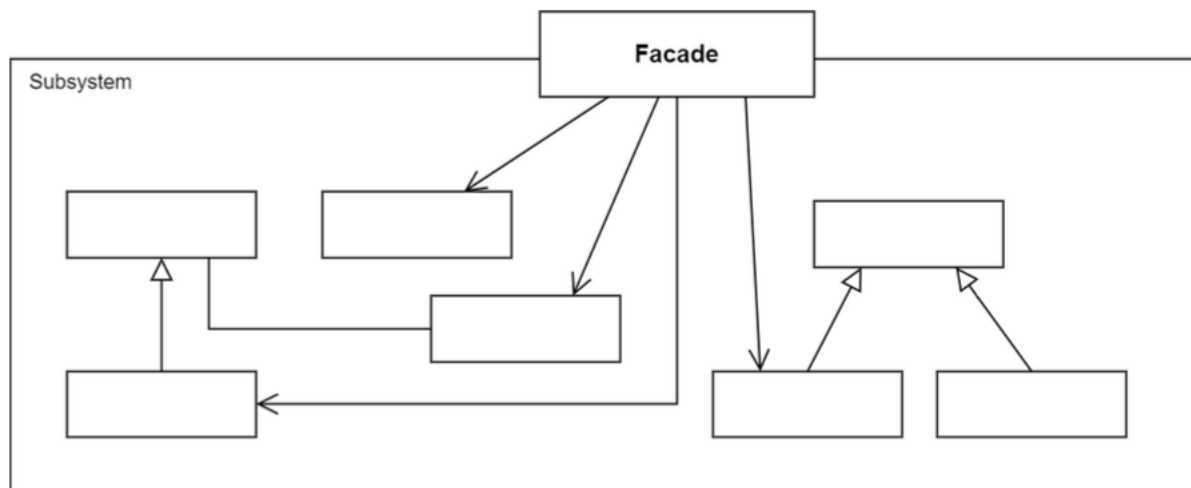


Рис-1. Структура патерну «Фасад»

Проблема: Ви розробляєте компонент, який дозволяє відправляти запити на різні типи endpoint, а також працює з протоколами HTTP та TCP/IP.

Прототип компонента вже працює, але структура класів вийшла досить складна, а при налаштуванні на різні протоколи мають використовуватися різні класи.

Інструкція для використання також виходить досить складна та заплутана.

Слід додати, так як інші системи будуть знати про внутрішню будову вашого компонента, а тому при спробі змінити внутрішню структуру в наступних версіях, вам прийдеться повідомляти про це всіх користувачів вашого компонента і для них перехід на наступну версію вашого компонента буде достатньо складним.

Рішення: Тут краще використати патерн фасад. А саме, в даному випадку, створити один клас, наприклад, `InternetClient`, та набір методів у нього, які будуть використовуватися для налаштування цього підключення, та його подальшого використання. Цей клас єдиний буде позначено як `public`, і тільки його будуть бачити ваші клієнти. Таким чином, з точки зору зовнішнього коду вони будуть працювати з набагато простішим інтерфейсом, а значить і інструкція використання буде значно простіша. Крім того, так як внутрішня структура повністю закрыта, то в наступних версіях ви можете її змінювати, як вам буде зручно, а з точки зору користувачів компонента, перехід на нову версію буде просто переключенням на нову версію бібліотеки.

Переваги та недоліки:

- + Інкапсуляція внутрішньої структури від клієнтського коду.
- + Спрощується інтерфейс для роботи з модулем закритим фасадом.
- Зниження гнучкості в налаштуванні та використанні програмного коду закритого фасадом.

Серед шаблонів Mediator, Bridge, Template Method і Facade, найкраще для мого застосунку підходить саме патерн Фасад, оскільки він вирішує ключову потребу програми, а тобто спрощення взаємодії між користувацьким інтерфейсом та внутрішньою логікою програвача.

Патерн Фасад дозволяє об'єднати ці підсистеми у єдиний спрощений інтерфейс, через який користувацькі дії (відтворення, пауза, додавання трека) виконуються без прямого доступу до внутрішніх модулів.

Хід роботи

У межах проєкту «Музичний програвач» було реалізовано основну логіку роботи з користувачами та аудіотреками.

Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.

Services > C# PlayerFacade.cs

```
1  using System;
2  using System.IO;
3  using System.Threading.Tasks;
4  using Microsoft.AspNetCore.Hosting;
5  using Microsoft.AspNetCore.Http;
6  using MusicPlayerWeb.Models;
7  using MusicPlayerWeb.Services.Commands;
8  using MusicPlayerWeb.Services.Playlists;
9
10 namespace MusicPlayerWeb.Services;
11
12 public class PlayerFacade
13 {
14     private readonly PlayerStateService _playerState;
15     private readonly PlayerCommandInvoker _commandInvoker;
16     private readonly ITrackService _tracks;
17     private readonly IPlaylistService _playlists;
18     private readonly IPlaylistHistoryService _history;
19     private readonly IWebHostEnvironment _env;
20
21     public PlayerFacade(
22         PlayerStateService playerState,
23         PlayerCommandInvoker commandInvoker,
24         ITrackService tracks,
25         IPlaylistService playlists,
26         IPlaylistHistoryService history,
27         IWebHostEnvironment env)
28     {
29         _playerState = playerState;
30         _commandInvoker = commandInvoker;
31         _tracks = tracks;
32         _playlists = playlists;
33         _history = history;
34         _env = env;
35     }
36
37     public PlayerOperationResult PlayTrack(string username, string title, string url)
38     {
39         if (string.IsNullOrEmpty(title) || string.IsNullOrEmpty(url))
40             return BuildPlayerResult(false, "Недостатньо даних для програвання треку.");
41
42         var command = new PlayTrackCommand(_playerState, title.Trim(), url);
43         _commandInvoker.Execute(command, username);
44
45         return BuildPlayerResult(true, $"Відтворюється \"{title.Trim()}\".");
46     }
47
48     public PlayerOperationResult PauseTrack(string username)
49     {
50         var command = new PauseTrackCommand(_playerState);
51         _commandInvoker.Execute(command, username);
52         return BuildPlayerResult(true, "Трек призупинено.");
53     }
54
55     public PlayerOperationResult StopTrack(string username)
56     {
57         var command = new StopTrackCommand(_playerState);
58         _commandInvoker.Execute(command, username);
59         return BuildPlayerResult(true, "Відтворення зупинено.");
60     }
61 }
```

```

61
62 public async Task<PlaylistOperationResult> AddTrackAsync(string username, Guid playlistId, string title, IFormFile? file)
63 {
64     if (playlistId == Guid.Empty)
65         return PlaylistOperationResult.Failure(Guid.Empty, "Вибери плейліст.");
66
67     if (string.IsNullOrEmpty(title))
68         return PlaylistOperationResult.Failure(playlistId, "Назва треку є обов'язковою.");
69
70     if (file == null || file.Length == 0)
71         return PlaylistOperationResult.Failure(playlistId, "Файл не вибраний.");
72
73     if (!_history.Backup(username, playlistId))
74         return PlaylistOperationResult.Failure(playlistId, "Плейліст не знайдено.");
75
76     var uploads = Path.Combine(_env.WebRootPath, "uploads");
77     Directory.CreateDirectory(uploads);
78
79     var safe = $"{Guid.NewGuid()}{Path.GetExtension(file.FileName)}";
80     var path = Path.Combine(uploads, safe);
81
82     await using (var stream = System.IO.File.Create(path))
83     {
84         await file.CopyToAsync(stream);
85     }
86
87     var rel = $"/uploads/{safe}";
88     var track = _tracks.AddForUser(username, title, safe, rel);
89
90     var added = _playlists.AddTrack(username, playlistId, track.Id);
91     if (!added)
92     {
93         _tracks.Delete(username, track.Id);
94         DeleteFileIfExists(path);
95         return PlaylistOperationResult.Failure(playlistId, "Не вдалося додати трек до плейліста.");
96     }
97
98     return PlaylistOperationResult.SuccessResult(playlistId, "Трек додано до плейліста.");
99 }
100
101 public PlaylistOperationResult RemoveTrack(string username, Guid playlistId, Guid trackId)
102 {
103     if (playlistId == Guid.Empty)
104         return PlaylistOperationResult.Failure(Guid.Empty, "Вибери плейліст.");
105
106     if (!_history.Backup(username, playlistId))
107         return PlaylistOperationResult.Failure(playlistId, "Плейліст не знайдено.");
108
109     var removed = _playlists.RemoveTrack(username, playlistId, trackId);
110     if (!removed)
111         return PlaylistOperationResult.Failure(playlistId, "Трек не знайдено в плейлісті.");
112
113     return PlaylistOperationResult.SuccessResult(playlistId, "Трек видалено з плейліста. Натисни \"Скасувати\", щоб повернути його.");
114 }
115
116 private PlayerOperationResult BuildPlayerResult(bool success, string? message = null)
117 {
118     return new PlayerOperationResult(_playerState.Snapshot, _commandInvoker.History, success, message);
119 }
120
121
122 private PlayerOperationResult BuildPlayerResult(bool success, string? message = null)
123 {
124     return new PlayerOperationResult(_playerState.Snapshot, _commandInvoker.History, success, message);
125 }
126
127 private static void DeleteFileIfExists(string path)
128 {
129     if (File.Exists(path))
130         File.Delete(path);
131 }

```

Рис-2-4. Services-PlayerFacade.cs

Controllers > C# PlayerController.cs

```
1  using Microsoft.AspNetCore.Authorization;
2  using Microsoft.AspNetCore.Mvc;
3  using MusicPlayerWeb.Models;
4  using MusicPlayerWeb.Services;
5
6  namespace MusicPlayerWeb.Controllers;
7
8  [Authorize]
9  [Route("[controller]/[action]")]
10 public class PlayerController : Controller
11 {
12     private readonly PlayerFacade _facade;
13
14     public PlayerController(PlayerFacade facade)
15     {
16         _facade = facade;
17     }
18
19     [HttpPost]
20     public IActionResult Play([FromBody] PlayTrackCommandRequest request)
21     {
22         if (request == null || string.IsNullOrEmpty(request.Title) || string.IsNullOrEmpty(request.Url))
23             return BadRequest("Недостаточно данных для проигрывания треку.");
24
25         var username = User.Identity?.Name ?? "unknown";
26         var result = _facade.PlayTrack(username, request.Title.Trim(), request.Url);
27         if (!result.Success)
28             return BadRequest(result.Message);
29         return Json(new { state = result.State, history = result.History, message = result.Message });
30     }
31
32     [HttpPost]
33     public IActionResult Pause()
34     {
35         var username = User.Identity?.Name ?? "unknown";
36         var result = _facade.PauseTrack(username);
37         return Json(new { state = result.State, history = result.History, message = result.Message });
38     }
39
40     [HttpPost]
41     public IActionResult Stop()
42     {
43         var username = User.Identity?.Name ?? "unknown";
44         var result = _facade.StopTrack(username);
45         return Json(new { state = result.State, history = result.History, message = result.Message });
46     }
47 }
48
49 public class PlayTrackCommandRequest
50 {
51     public string Title { get; set; } = string.Empty;
52     public string Url { get; set; } = string.Empty;
53 }
```

Рис-5. Controllers - PlayerController.cs

Controllers > C# TracksController.cs

```
1 using Microsoft.AspNetCore.Authorization;
2 using Microsoft.AspNetCore.Mvc;
3 using Microsoft.AspNetCore.Http;
4 using MusicPlayerWeb.Services;
5
6 namespace MusicPlayerWeb.Controllers;
7
8 [Authorize]
9 public class TracksController : Controller
10 {
11     private readonly PlayerFacade _facade;
12
13     public TracksController(PlayerFacade facade)
14     {
15         _facade = facade;
16     }
17
18     [HttpPost]
19     public async Task<IActionResult> Add(string title, IFormFile file, Guid playlistId)
20     {
21         var username = User.Identity!.Name!;
22         var result = await _facade.AddTrackAsync(username, playlistId, title, file);
23         if (!result.Success)
24         {
25             TempData["Error"] = result.Message;
26         }
27         else if (!string.IsNullOrEmpty(result.Message))
28         {
29             TempData["Info"] = result.Message;
30         }
31
32         return RedirectToAction("Index", "Home", new { playlistId = result.PlaylistId == Guid.Empty ? playlistId : result.PlaylistId });
33     }
34
35     [HttpPost]
36     public IActionResult Delete(Guid id, Guid playlistId)
37     {
38         var username = User.Identity!.Name!;
39         var result = _facade.RemoveTrack(username, playlistId, id);
40         if (!result.Success)
41         {
42             TempData["Error"] = result.Message;
43         }
44         else if (!string.IsNullOrEmpty(result.Message))
45         {
46             TempData["Info"] = result.Message;
47         }
48         return RedirectToAction("Index", "Home", new { playlistId });
49     }
50 }
```

Рис-6. Controllers - TracksController.cs

Models > C# PlayerOperationResult.cs

```
1 using System.Collections.Generic;
2
3 namespace MusicPlayerWeb.Models;
4
5 public class PlayerOperationResult
6 {
7     public PlayerOperationResult(PlayerStateSnapshot state, IReadOnlyCollection<PlayerCommandHistoryItem> history, bool success, string? message = null)
8     {
9         State = state;
10        History = history;
11        Success = success;
12        Message = message;
13    }
14
15    public PlayerStateSnapshot State { get; }
16    public IReadOnlyCollection<PlayerCommandHistoryItem> History { get; }
17    public bool Success { get; }
18    public string? Message { get; }
19 }
20
```

Рис-7. Models – PlayerOperationResult.cs


```

Models > C# PlaylistOperationResult.cs
1  using System;
2
3  namespace MusicPlayerWeb.Models;
4
5  public class PlaylistOperationResult
6  {
7      public PlaylistOperationResult(bool success, Guid playlistId, string? message = null)
8      {
9          Success = success;
10         PlaylistId = playlistId;
11         Message = message;
12     }
13
14     public bool Success { get; }
15     public Guid PlaylistId { get; }
16     public string? Message { get; }
17
18     public static PlaylistOperationResult SuccessResult(Guid playlistId, string? message = null)
19     => new PlaylistOperationResult(true, playlistId, message);
20
21     public static PlaylistOperationResult Failure(Guid playlistId, string message)
22     => new PlaylistOperationResult(false, playlistId, message);
23 }
24

```

Рис-8. Models – PlaylistOperationResult.cs

Реалізувати один з розглянутих шаблонів за обраною темою.

Для спрощення взаємодії інтерфейсу користувача з внутрішніми підсистемами музичного програвача було використано патерн Facade.

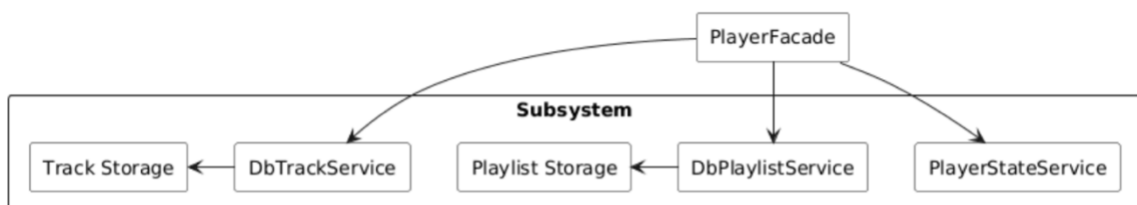


Рис-9. Структура патерну Facade

1. Клас PlayerFacade

Клас PlayerFacade, який виступає єдиною точкою взаємодії для зовнішніх компонентів системи. Саме через нього відбувається виклик операцій, пов'язаних із відтворенням треків, керуванням списками відтворення та доступом до збережених аудіофайлів.

Нижче знаходиться прямокутна область, позначена як Subsystem, яка представляє внутрішню складну частину системи. Всередині розміщено підсистеми, приховані від клієнтського коду:

2. DbTrackService

Відповідає за додавання, зберігання та видалення треків;

3. Track Storage

Відображає місце, де фізично зберігаються файли треків; 4. PlaylistHistoryService - координатор відновлення

4. DbPlaylistService

Керує створенням, перейменуванням, видаленням та наповненням плейлістів;

5. Playlist Storage

Представляє збереження структур плейлістів;

6. PlayerStateService

Відповідає за операції відтворення, паузи та зупинки.

Services > C# PlayerFacade.cs

```
1  using System;
2  using System.IO;
3  using System.Threading.Tasks;
4  using Microsoft.AspNetCore.Hosting;
5  using Microsoft.AspNetCore.Http;
6  using MusicPlayerWeb.Models;
7  using MusicPlayerWeb.Services.Commands;
8  using MusicPlayerWeb.Services.Playlists;
9
10 namespace MusicPlayerWeb.Services;
11
12 public class PlayerFacade
13 {
14     private readonly PlayerStateService _playerState;
15     private readonly PlayerCommandInvoker _commandInvoker;
16     private readonly ITrackService _tracks;
17     private readonly IPlaylistService _playlists;
18     private readonly IPlaylistHistoryService _history;
19     private readonly IWebHostEnvironment _env;
20
21     public PlayerFacade(
22         PlayerStateService playerState,
23         PlayerCommandInvoker commandInvoker,
24         ITrackService tracks,
25         IPlaylistService playlists,
26         IPlaylistHistoryService history,
27         IWebHostEnvironment env)
28     {
29         _playerState = playerState;
30         _commandInvoker = commandInvoker;
31         _tracks = tracks;
32         _playlists = playlists;
33         _history = history;
34         _env = env;
35     }
36
37     public PlayerOperationResult PlayTrack(string username, string title, string url)
38     {
39         if (string.IsNullOrEmpty(title) || string.IsNullOrEmpty(url))
40             return BuildPlayerResult(false, "Недостатньо даних для програвання треку.");
41
42         var command = new PlayTrackCommand(_playerState, title.Trim(), url);
43         _commandInvoker.Execute(command, username);
44
45         return BuildPlayerResult(true, $"Відтворюється \"{title.Trim()}\".");
46     }
47
48     public PlayerOperationResult PauseTrack(string username)
49     {
50         var command = new PauseTrackCommand(_playerState);
51         _commandInvoker.Execute(command, username);
52         return BuildPlayerResult(true, "Трек призупинено.");
53     }
54
55     public PlayerOperationResult StopTrack(string username)
56     {
57         var command = new StopTrackCommand(_playerState);
58         _commandInvoker.Execute(command, username);
59         return BuildPlayerResult(true, "Відтворення зупинено.");
60     }
61 }
```

```

public async Task<PlaylistOperationResult> AddTrackAsync(string username, Guid playlistId, string title, IFormFile? file)
{
    if (playlistId == Guid.Empty)
        return PlaylistOperationResult.Failure(Guid.Empty, "Вибери плейліст.");

    if (string.IsNullOrWhiteSpace(title))
        return PlaylistOperationResult.Failure(playlistId, "Назва треку є обов'язковою.");

    if (file == null || file.Length == 0)
        return PlaylistOperationResult.Failure(playlistId, "Файл не вибраний.");

    if (!_history.Backup(username, playlistId))
        return PlaylistOperationResult.Failure(playlistId, "Плейліст не знайдено.");

    var uploads = Path.Combine(_env.WebRootPath, "uploads");
    Directory.CreateDirectory(uploads);

    var safe = $"{Guid.NewGuid()}_{Path.GetExtension(file.FileName)}";
    var path = Path.Combine(uploads, safe);

    await using (var stream = System.IO.File.Create(path))
    {
        await file.CopyToAsync(stream);
    }

    var rel = $"uploads/{safe}";
    var track = _tracks.AddForUser(username, title, safe, rel);

    var added = _playlists.AddTrack(username, playlistId, track.Id);
    if (!added)
    {
        _tracks.Delete(username, track.Id);
        DeleteFileIfExists(path);
        return PlaylistOperationResult.Failure(playlistId, "Не вдалося додати трек до плейліста.");
    }

    return PlaylistOperationResult.SuccessResult(playlistId, "Трек додано до плейліста.");
}

public PlaylistOperationResult RemoveTrack(string username, Guid playlistId, Guid trackId)
{
    if (playlistId == Guid.Empty)
        return PlaylistOperationResult.Failure(Guid.Empty, "Вибери плейліст.");

    if (!_history.Backup(username, playlistId))
        return PlaylistOperationResult.Failure(playlistId, "Плейліст не знайдено.");

    var removed = _playlists.RemoveTrack(username, playlistId, trackId);
    if (!removed)
        return PlaylistOperationResult.Failure(playlistId, "Трек не знайдено в плейлісті.");

    return PlaylistOperationResult.SuccessResult(playlistId, "Трек видалено з плейліста. Натисни \"Скасувати\", щоб повернути його.");
}

private PlayerOperationResult BuildPlayerResult(bool success, string? message = null)
{
    return new PlayerOperationResult(_playerState.Snapshot, _commandInvoker.History, success, message);
}

```

```

120
121     private static void DeleteFileIfExists(string path)
122     {
123         if (File.Exists(path))
124             File.Delete(path);
125     }
126 }
127

```

Рис-10. Services - PlayerFacade.cs

```

Controllers > C# PlayerController.cs
1  using Microsoft.AspNetCore.Authorization;
2  using Microsoft.AspNetCore.Mvc;
3  using MusicPlayerWeb.Models;
4  using MusicPlayerWeb.Services;
5
6  namespace MusicPlayerWeb.Controllers;
7
8  [Authorize]
9  [Route("[controller]/[action]")]
10 public class PlayerController : Controller
11 {
12     private readonly PlayerFacade _facade;
13
14     public PlayerController(PlayerFacade facade)
15     {
16         _facade = facade;
17     }
18
19     [HttpPost]
20     public IActionResult Play([FromBody] PlayTrackCommandRequest request)
21     {
22         if (request == null || string.IsNullOrEmpty(request.Title) || string.IsNullOrEmpty(request.Url))
23             return BadRequest("Недостаточно данных для проигрывания треку.");
24
25         var username = User.Identity?.Name ?? "unknown";
26         var result = _facade.PlayTrack(username, request.Title.Trim(), request.Url);
27         if (!result.Success)
28             return BadRequest(result.Message);
29         return Json(new { state = result.State, history = result.History, message = result.Message });
30     }
31
32     [HttpPost]
33     public IActionResult Pause()
34     {
35         var username = User.Identity?.Name ?? "unknown";
36         var result = _facade.PauseTrack(username);
37         return Json(new { state = result.State, history = result.History, message = result.Message });
38     }
39
40     [HttpPost]
41     public IActionResult Stop()
42     {
43         var username = User.Identity?.Name ?? "unknown";
44         var result = _facade.StopTrack(username);
45         return Json(new { state = result.State, history = result.History, message = result.Message });
46     }
47 }
48
49 public class PlayTrackCommandRequest
50 {
51     public string Title { get; set; } = string.Empty;
52     public string Url { get; set; } = string.Empty;
53 }
54

```

Рис-11. Contorrollers – PlayerController.cs

```

Controllers > C# TracksController.cs
1  using Microsoft.AspNetCore.Authorization;
2  using Microsoft.AspNetCore.Mvc;
3  using Microsoft.AspNetCore.Http;
4  using MusicPlayerWeb.Services;
5
6  namespace MusicPlayerWeb.Controllers;
7
8  [Authorize]
9  public class TracksController : Controller
10 {
11     private readonly PlayerFacade _facade;
12
13     public TracksController(PlayerFacade facade)
14     {
15         _facade = facade;
16     }
17
18     [HttpPost]
19     public async Task<IActionResult> Add(string title, IFormFile file, Guid playlistId)
20     {
21         var username = User.Identity!.Name!;
22         var result = await _facade.AddTrackAsync(username, playlistId, title, file);
23         if (!result.Success)
24         {
25             TempData["Error"] = result.Message;
26         }
27         else if (!string.IsNullOrEmpty(result.Message))
28         {
29             TempData["Info"] = result.Message;
30         }
31
32         return RedirectToAction("Index", "Home", new { playlistId = result.PlaylistId == Guid.Empty ? playlistId : result.PlaylistId });
33     }
34
35     [HttpPost]
36     public IActionResult Delete(Guid id, Guid playlistId)
37     {
38         var username = User.Identity!.Name!;
39         var result = _facade.RemoveTrack(username, playlistId, id);
40         if (!result.Success)
41         {
42             TempData["Error"] = result.Message;
43         }
44         else if (!string.IsNullOrEmpty(result.Message))
45         {
46             TempData["Info"] = result.Message;
47         }
48
49         return RedirectToAction("Index", "Home", new { playlistId });
50     }
51 }

```

Рис-12. Contorollers – PlayerController.cs

Services > Playlists > C# PlaylistHistoryCaretaker.cs

```
1  using System.Collections.Generic;
2
3  namespace MusicPlayerWeb.Services.Playlists;
4
5  public class PlaylistHistoryCaretaker
6  {
7      private readonly Stack<PlaylistMemento> _history = new();
8      private readonly object _sync = new();
9
10     public void Push(PlaylistMemento memento)
11     {
12         lock (_sync)
13         {
14             _history.Push(memento);
15         }
16     }
17
18     public PlaylistMemento? Pop()
19     {
20         lock (_sync)
21         {
22             if (_history.Count == 0)
23                 return null;
24             return _history.Pop();
25         }
26     }
27
28     public bool HasHistory
29     {
30         get
31         {
32             lock (_sync)
33             {
34                 return _history.Count > 0;
35             }
36         }
37     }
38 }
39 }
```

Рис-8. Services-Playlists-PlaylistHistoryCaretaker.cs

```

1  using System;
2  using System.Linq;
3  using Microsoft.Extensions.Caching.Memory;
4  using MusicPlayerWeb.Services;
5
6  namespace MusicPlayerWeb.Services.Playlists;
7
8  public class PlaylistHistoryService : IPlaylistHistoryService
9  {
10     private readonly IPlaylistService _playlists;
11     private readonly IMemoryCache _cache;
12     private static readonly TimeSpan HistoryLifetime = TimeSpan.FromMinutes(30);
13
14     public PlaylistHistoryService(IPlaylistService playlists, IMemoryCache cache)
15     {
16         _playlists = playlists;
17         _cache = cache;
18     }
19
20     public bool Backup(string username, Guid playlistId)
21     {
22         var playlist = _playlists.GetById(username, playlistId);
23         if (playlist == null)
24             return false;
25
26         var tracks = _playlists.GetTracks(username, playlistId).ToList();
27         var originator = new PlaylistOriginator();
28         originator.Load(playlist, tracks);
29
30         var caretaker = _cache.GetOrCreate(BuildKey(username, playlistId), entry =>
31         {
32             entry.SlidingExpiration = HistoryLifetime;
33             return new PlaylistHistoryCaretaker();
34         });
35
36         if (caretaker == null)
37             return false;
38
39         caretaker.Push(originator.Save());
40         return true;
41     }
42
43     public bool Undo(string username, Guid playlistId)
44     {
45         if (!_cache.TryGetValue(BuildKey(username, playlistId), out PlaylistHistoryCaretaker? caretaker) || caretaker == null)
46             return false;
47
48         var snapshot = caretaker.Pop();
49         if (snapshot == null)
50             return false;
51
52         var originator = new PlaylistOriginator();
53         originator.Restore(snapshot);
54         return ApplyState(username, originator);
55     }
56 }

```



```

56
57 public bool HasHistory(string username, Guid playlistId)
58 {
59     if (!_cache.TryGetValue(BuildKey(username, playlistId), out PlaylistHistoryCaretaker? caretaker) || caretaker == null)
60         return false;
61
62     return caretaker.HasHistory;
63 }
64
65 private bool ApplyState(string username, PlaylistOriginator originator)
66 {
67     var restored = _playlists.Restore(username, originator.PlaylistId, originator.Title);
68     if (!restored)
69         return false;
70
71     var existing = _playlists.GetTracks(username, originator.PlaylistId)
72         .Select(t => t.Id)
73         .ToList();
74
75     foreach (var trackId in existing)
76     {
77         _playlists.RemoveTrack(username, originator.PlaylistId, trackId);
78     }
79
80     foreach (var trackId in originator.TrackIds)
81     {
82         _playlists.AddTrack(username, originator.PlaylistId, trackId);
83     }
84
85     return true;
86 }
87
88 private static string BuildKey(string username, Guid playlistId)
89 {
90     return $"playlist-history:{username}:{playlistId}";
91 }
92 }
93

```

Рис-9-10. Services-Playlists-PlaylistHistoryService.cs

Services > Playlists > C# IPlaylistHistoryService.cs

```

1 namespace MusicPlayerWeb.Services.Playlists;
2
3 public interface IPlaylistHistoryService
4 {
5     bool Backup(string username, Guid playlistId);
6     bool Undo(string username, Guid playlistId);
7     bool HasHistory(string username, Guid playlistId);
8 }
9

```

Рис-11. Services-Playlists-IPlaylistHistoryService.cs

Controllers > C# PlaylistsController.cs

```
1 using Microsoft.AspNetCore.Authorization;
2 using Microsoft.AspNetCore.Mvc;
3 using MusicPlayerWeb.Models;
4 using MusicPlayerWeb.Services;
5 using MusicPlayerWeb.Services.Playlists;
6 using System.Linq;
7
8 namespace MusicPlayerWeb.Controllers;
9
10 [Authorize]
11 public class PlaylistsController : Controller
12 {
13     private readonly IPlaylistService _playlists;
14     private readonly IPlaylistHistoryService _history;
15
16     public PlaylistsController(IPlaylistService playlists, IPlaylistHistoryService history)
17     {
18         _playlists = playlists;
19         _history = history;
20     }
21
22     [HttpPost]
23     public IActionResult Create(string title)
24     {
25         var username = User.Identity!.Name!;
26
27         if (string.IsNullOrEmpty(title))
28         {
29             TempData["Error"] = "Назва плейліста є обов'язковою.";
30             return RedirectToAction("Index", "Home");
31         }
32
33         var playlist = _playlists.Create(username, title);
34         return RedirectToAction("Index", "Home", new { playlistId = playlist.Id });
35     }
36
37     [HttpPost]
38     public IActionResult Rename(Guid playlistId, string newTitle)
39     {
40         var username = User.Identity!.Name!;
41
42         if (string.IsNullOrEmpty(newTitle))
43         {
44             TempData["Error"] = "Нова назва не може бути порожньою.";
45             return RedirectToAction("Index", "Home", new { playlistId });
46         }
47
48         if (!_history.Backup(username, playlistId))
49         {
50             TempData["Error"] = "Плейліст не знайдено для створення знімка стану.";
51             return RedirectToAction("Index", "Home", new { playlistId });
52         }
53
54         var ok = _playlists.Rename(username, playlistId, newTitle);
55     }
56 }
```

```

55
56         if (!ok)
57             TempData["Error"] = "Плейліст не знайдено.";
58
59         return RedirectToAction("Index", "Home", new { playlistId });
60     }
61
62     [HttpPost]
63     public IActionResult Delete(Guid playlistId)
64     {
65         var username = User.Identity!.Name!;
66
67         if (!_history.Backup(username, playlistId))
68         {
69             TempData["Error"] = "Плейліст не знайдено.";
70             return RedirectToAction("Index", "Home");
71         }
72
73         var ok = _playlists.Delete(username, playlistId);
74
75         if (!ok)
76         {
77             TempData["Error"] = "Плейліст не знайдено.";
78             return RedirectToAction("Index", "Home");
79         }
80
81         TempData["Info"] = "Плейліст видалено. Натисни \"Скасувати\", щоб повернути його.";
82         return RedirectToAction("Index", "Home", new { playlistId });
83     }
84
85     [HttpPost]
86     public IActionResult Undo(Guid playlistId)
87     {
88         var username = User.Identity!.Name!;
89         if (!_history.Undo(username, playlistId))
90         {
91             TempData["Error"] = "Немає попереднього стану для відновлення.";
92         }
93         else
94         {
95             TempData["Info"] = "Стан плейліста відновлено.";
96         }
97
98         return RedirectToAction("Index", "Home", new { playlistId });
99     }
100
101     [HttpGet]

```

Рис-12-13. Controllers-PlaylistController.cs

Controllers > C# TracksController.cs

```
1 using Microsoft.AspNetCore.Authorization;
2 using Microsoft.AspNetCore.Mvc;
3 using MusicPlayerWeb.Services;
4 using MusicPlayerWeb.Services.Playlists;
5
6 namespace MusicPlayerWeb.Controllers;
7
8 [Authorize]
9 public class TracksController : Controller
10 {
11     private readonly IWebHostEnvironment _env;
12     private readonly ITrackService _tracks;
13     private readonly IPlaylistService _playlists;
14     private readonly IPlaylistHistoryService _history;
15
16     public TracksController(IWebHostEnvironment env, ITrackService tracks, IPlaylistService playlists, IPlaylistHistoryService history)
17     {
18         _env = env;
19         _tracks = tracks;
20         _playlists = playlists;
21         _history = history;
22     }
23
24     [HttpPost]
25     public async Task<ActionResult> Add(string title, IFormFile file, Guid playlistId)
26     {
27         var username = User.Identity!.Name!;
28
29         if (playlistId == Guid.Empty)
30         {
31             TempData["Error"] = "Вибери плейліст.";
32             return RedirectToAction("Index", "Home");
33         }
34
35         if (string.IsNullOrWhiteSpace(title))
36         {
37             TempData["Error"] = "Назва обов'язкова.";
38             return RedirectToAction("Index", "Home", new { playlistId });
39         }
40
41         if (file == null || file.Length == 0)
42         {
43             TempData["Error"] = "Файл не вибраний.";
44             return RedirectToAction("Index", "Home", new { playlistId });
45         }
46
47         if (!_history.Backup(username, playlistId))
48         {
49             TempData["Error"] = "Плейліст не знайдено.";
50             return RedirectToAction("Index", "Home", new { playlistId });
51         }
52
53         var uploads = Path.Combine(_env.WebRootPath, "uploads");
54         Directory.CreateDirectory(uploads);
55     }
56 }
```

```

56     var safe = $"{Guid.NewGuid()}_{Path.GetExtension(file.FileName)}";
57     var path = Path.Combine(uploads, safe);
58
59     await using (var stream = System.IO.File.Create(path))
60     {
61         await file.CopyToAsync(stream);
62     }
63
64     var rel = $"/uploads/{safe}";
65     var track = _tracks.AddForUser(username, title, safe, rel);
66
67     var added = _playlists.AddTrack(username, playlistId, track.Id);
68     if (!added)
69     {
70         TempData["Error"] = "Не вдалося додати трек до плейліста.";
71     }
72
73     return RedirectToAction("Index", "Home", new { playlistId });
74 }
75
76 [HttpPost]
77 public IActionResult Delete(Guid id, Guid playlistId)
78 {
79     var username = User.Identity!.Name!;
80     if (!_history.Backup(username, playlistId))
81     {
82         TempData["Error"] = "Плейліст не знайдено.";
83         return RedirectToAction("Index", "Home", new { playlistId });
84     }
85
86     var removed = _playlists.RemoveTrack(username, playlistId, id);
87     if (!removed)
88     {
89         TempData["Error"] = "Трек не знайдено в плейлісті.";
90     }
91     return RedirectToAction("Index", "Home", new { playlistId });
92 }

```

Рис-14-15. Controllers-TrackController.cs

Views > Home > @ Index.cshtml

```
1  @model MusicPlayerWeb.Models.PlaylistPageViewModel
2
3  <h2>Музичний програвач</h2>
4
5  @if (TempData["Error"] != null)
6  {
7      <div class="alert">@TempData["Error"]</div>
8  }
9  @if (TempData["Info"] != null)
10 {
11     <div class="alert info">@TempData["Info"]</div>
12 }
13
14 <div class="layout">
15
16     <aside class="sidebar card-panel">
17         <h3>Мої плейлісти</h3>
18
19         @foreach (var p in Model.Playlists)
20         {
21             var active = Model.SelectedPlaylistId == p.Id ? "active" : "";
22             <div class="playlist-item @active">
23                 <div class="playlist-link">
24                     <a asp-controller="Home"
25                        asp-action="Index"
26                        asp-route-playlistId="@p.Id">@p.Title</a>
27                 </div>
28                 <form asp-controller="Playlists" asp-action="Rename" method="post" class="inline-form">
29                     <input type="hidden" name="playlistId" value="@p.Id" />
30                     <input type="text" name="newTitle" value="@p.Title" required />
31                     <button title="Перейменувати">Перейменувати</button>
32                 </form>
33                 <form asp-controller="Playlists" asp-action="Delete" method="post" class="inline-form">
34                     <input type="hidden" name="playlistId" value="@p.Id" />
35                     <button class="danger" title="Видалити">✖</button>
36                 </form>
37             </div>
38         }
39
40         <h4>Новий плейліст</h4>
41         <form asp-controller="Playlists" asp-action="Create" method="post" class="new-playlist-form">
42             <input type="text" name="title" required />
43             <button>Створити</button>
44         </form>
45     </aside>
46
```

```

47 <section class="main">
48
49     @if (!Model.SelectedPlaylistId.HasValue)
50     {
51         <p>Вибери плейліст.</p>
52         return;
53     }
54
55     <h3>Плейліст: @Model.SelectedPlaylistTitle</h3>
56     @if (Model.SelectedPlaylistId.HasValue)
57     {
58         <div class="playlist-undo">
59             <form asp-controller="Playlists" asp-action="Undo" method="post" class="inline-form">
60                 <input type="hidden" name="playlistId" value="@Model.SelectedPlaylistId" />
61                 @if (Model.CanUndoPlaylistChanges)
62                 {
63                     <button type="submit">↶ Скасувати останню дію</button>
64                 }
65                 else
66                 {
67                     <button type="submit" disabled title="Немає попереднього стану">↶ Скасувати останню дію</button>
68                 }
69             </form>
70         </div>
71     }
72
73     <section class="uploader card-panel">
74         <form asp-controller="Tracks" asp-action="Add" enctype="multipart/form-data" method="post">
75
76             <label>До якого плейліста додати?</label>
77             <select name="playlistId" required>
78                 @foreach (var p in Model.Playlists)
79                 {
80                     <option value="@p.Id" selected="@((p.Id == Model.SelectedPlaylistId))">
81                         @p.Title
82                     </option>
83                 }
84             </select>
85
86             <label>Назва треку</label>
87             <input name="title" required />
88
89             <label>Файл</label>
90             <input type="file" name="file" accept="audio/*" required />
91
92             <button type="submit">Додати трек</button>
93         </form>
94     </section>
95

```



```

96     <section class="list card-panel">
97         <h3>Треки</h3>
98
99         @if (!Model.Tracks.Any())
100         {
101             <p>У цьому плейлісті ще немає треків.</p>
102         }
103         else
104         {
105             <table class="tracks">
106                 <thead>
107                     <tr><th>Назва</th><th>Дія</th></tr>
108                 </thead>
109                 <tbody>
110                     @foreach (var t in Model.Tracks)
111                     {
112                         <tr data-url="@t.RelativeUrl" data-title="@t.Title">
113                             <td>@t.Title</td>
114                             <td>
115                                 <button class="playThis">>></button>
116
117                                 <form asp-controller="Tracks" asp-action="Delete" method="post" style="display:inline">
118                                     <input type="hidden" name="id" value="@t.Id" />
119                                     <input type="hidden" name="playlistId" value="@Model.SelectedPlaylistId" />
120                                     <button class="danger">Видалити</button>
121                                 </form>
122                             </td>
123                         </tr>
124                     }
125                 </tbody>
126             </table>
127         }
128     </section>
129
130     <section class="player card-panel">
131         <audio id="player" controls preload="metadata" style="width:100%">
132             <source id="playerSource" src="" type="audio/mpeg" />
133             Тер аудіо не підтримується.
134         </audio>
135         <div id="nowPlaying"></div>
136         <div class="controls">
137             <button id="btnPlay">>> Відтворити</button>
138             <button id="btnPause">⏸ Пауза</button>
139             <button id="btnStop" class="danger">■ Стоп</button>
140             <button id="btnBack">⏮ -10с</button>
141             <button id="btnFwd">⏭ +10с</button>
142             <button id="btnShuffle">🔀 Shuffle</button>
143             <button id="btnRepeat">🔁 Repeat</button>
144         </div>
145
146         <div>
147             <section class="equalizer">
148                 <h4>Еквалайзер</h4>
149                 <label for="eqBass">Низькі частоти</label>
150                 <input type="range" id="eqBass" min="-15" max="15" value="0" />
151                 <label for="eqTreble">Високі частоти</label>
152                 <input type="range" id="eqTreble" min="-15" max="15" value="0" />
153             </section>
154         </div>
155
156     </div>
157
158     <script src="~/js/player.js"></script>
159

```

Рис-16-19. Views-Home-Index.cshtml'

Реалізувати не менше 3-х класів відповідно до обраної теми.

Services > C# PlayerStateService.cs

```
1  using MusicPlayerWeb.Models;
2
3  namespace MusicPlayerWeb.Services;
4
5  public class PlayerStateService
6  {
7      private readonly object _sync = new();
8      private PlayerStateSnapshot _snapshot = PlayerStateSnapshot.Empty;
9
10     public PlayerStateSnapshot Snapshot
11     {
12         get
13         {
14             lock (_sync)
15             {
16                 return _snapshot;
17             }
18         }
19     }
20
21     public void Play(string title, string url)
22     {
23         lock (_sync)
24         {
25             _snapshot = PlayerStateSnapshot.Create(title, url, PlayerPlaybackState.Playing);
26         }
27     }
28
29     public void Pause()
30     {
31         lock (_sync)
32         {
33             if (!_snapshot.HasTrack)
34                 return;
35
36             _snapshot = PlayerStateSnapshot.Create(_snapshot.Title, _snapshot.Url, PlayerPlaybackState.Paused);
37         }
38     }
39
40     public void Stop()
41     {
42         lock (_sync)
43         {
44             _snapshot = PlayerStateSnapshot.Create(null, null, PlayerPlaybackState.Stopped);
45         }
46     }
47 }
```

Рис-20. Services – PlayerStateService.cs

Services > Commands > C# PlayerCommandInvoker.cs

```
1  using MusicPlayerWeb.Models;
2
3  namespace MusicPlayerWeb.Services.Commands;
4
5  public class PlayerCommandInvoker
6  {
7      private const int MaxHistory = 20;
8      private readonly List<PlayerCommandHistoryItem> _history = new();
9
10     public IReadOnlyCollection<PlayerCommandHistoryItem> History
11     {
12         get
13         {
14             lock (_history)
15             {
16                 return _history.ToArray();
17             }
18         }
19     }
20
21     public void Execute(IPlayerCommand command, string username)
22     {
23         command.Execute();
24         AddToHistory(command.Name, username);
25     }
26
27     private void AddToHistory(string commandName, string username)
28     {
29         lock (_history)
30         {
31             _history.Add(new PlayerCommandHistoryItem
32             {
33                 CommandName = commandName,
34                 ExecutedBy = username,
35                 ExecutedAtUtc = DateTime.UtcNow
36             });
37
38             if (_history.Count > MaxHistory)
39             {
40                 _history.RemoveRange(0, _history.Count - MaxHistory);
41             }
42         }
43     }
44 }
```

Рис-21. Services – Commands - PlayerCommandInvoker.cs

```
1  using System;
2  using System.Linq;
3  using Microsoft.Extensions.Caching.Memory;
4  using MusicPlayerWeb.Services;
5
6  namespace MusicPlayerWeb.Services.Playlists;
7
8  public class PlaylistHistoryService : IPlaylistHistoryService
9  {
10     private readonly IPlaylistService _playlists;
11     private readonly IMemoryCache _cache;
12     private static readonly TimeSpan HistoryLifetime = TimeSpan.FromMinutes(30);
13
14     public PlaylistHistoryService(IPlaylistService playlists, IMemoryCache cache)
15     {
16         _playlists = playlists;
17         _cache = cache;
18     }
19
20     public bool Backup(string username, Guid playlistId)
21     {
22         var playlist = _playlists.GetById(username, playlistId);
23         if (playlist == null)
24             return false;
25
26         var tracks = _playlists.GetTracks(username, playlistId).ToList();
27         var originator = new PlaylistOriginator();
28         originator.Load(playlist, tracks);
29
30         var caretaker = _cache.GetOrCreate(BuildKey(username, playlistId), entry =>
31         {
32             entry.SlidingExpiration = HistoryLifetime;
33             return new PlaylistHistoryCaretaker();
34         });
35
36         if (caretaker == null)
37             return false;
38
39         caretaker.Push(originator.Save());
40         return true;
41     }
42
43     public bool Undo(string username, Guid playlistId)
44     {
45         if (!_cache.TryGetValue(BuildKey(username, playlistId), out PlaylistHistoryCaretaker? caretaker) || caretaker == null)
46             return false;
47
48         var snapshot = caretaker.Pop();
49         if (snapshot == null)
50             return false;
51
52         var originator = new PlaylistOriginator();
53         originator.Restore(snapshot);
54         return ApplyState(username, originator);
55     }
56 }
```

```

50
57 public bool HasHistory(string username, Guid playlistId)
58 {
59     if (!_cache.TryGetValue(BuildKey(username, playlistId), out PlaylistHistoryCaretaker? caretaker) || caretaker == null)
60         return false;
61
62     return caretaker.HasHistory;
63 }
64
65 private bool ApplyState(string username, PlaylistOriginator originator)
66 {
67     var restored = _playlists.Restore(username, originator.PlaylistId, originator.Title);
68     if (!restored)
69         return false;
70
71     var existing = _playlists.GetTracks(username, originator.PlaylistId)
72         .Select(t => t.Id)
73         .ToList();
74
75     foreach (var trackId in existing)
76     {
77         _playlists.RemoveTrack(username, originator.PlaylistId, trackId);
78     }
79
80     foreach (var trackId in originator.TrackIds)
81     {
82         _playlists.AddTrack(username, originator.PlaylistId, trackId);
83     }
84
85     return true;
86 }
87
88 private static string BuildKey(string username, Guid playlistId)
89 {
90     return $"playlist-history:{username}:{playlistId}";
91 }
92 }
93

```

Рис-22-23. Services – Playlists – PlaylistHistoryService.cs

Висновок:

У ході виконання лабораторної роботи я опрацювала шаблони проектування Abstract Factory, Factory Method, Memento, Observer та Decorator та розібралася з їхнім призначенням, структурою та особливостями використання. На прикладі свого веб-застосунку «Музичний програвач» я реалізувала патерн Memento для скасування змін у плейлістах, що дозволило відновлювати попередній стан без втручання у внутрішню логіку та структуру даних. Під час роботи я переконалася, що використання шаблонів робить систему гнучкішою, легшою для супроводу та розширення, а також допомагає уникати дублювання коду. Виконання цієї лабораторної роботи дало мені можливість не лише зрозуміти теорію патернів, але й застосувати їх на практиці у власному проєкті, що значно поглибило мої навички та впевненість у розробці програмного забезпечення.

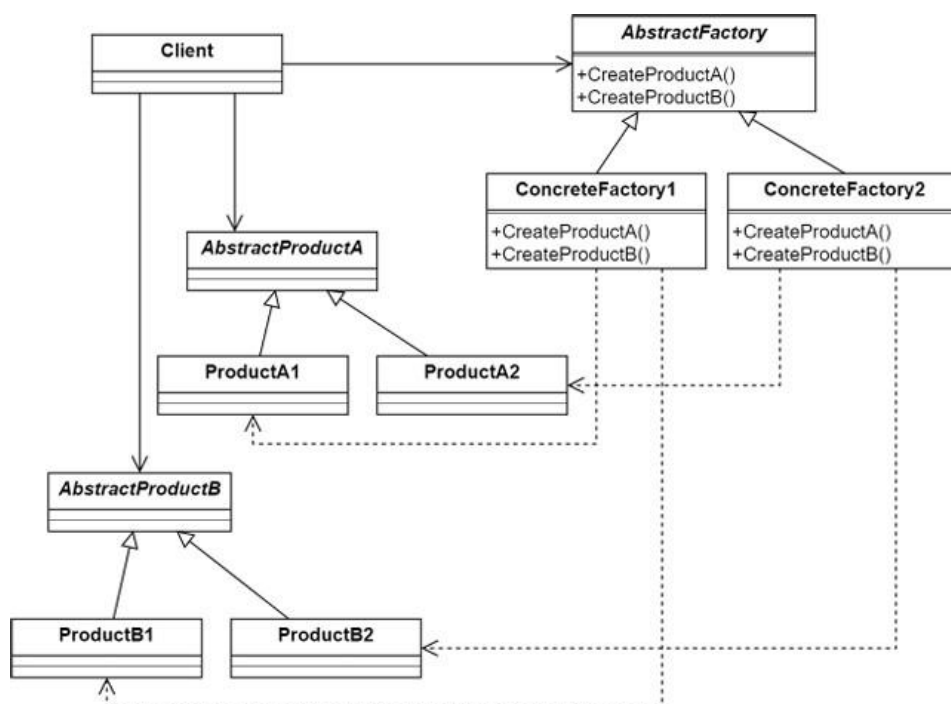
Відповіді на контрольні запитання:

1. Призначення шаблону «Абстрактна фабрика»

Шаблон «Абстрактна фабрика» (Abstract Factory) призначений для створення сімейств пов'язаних об'єктів без прив'язки до конкретних класів.

Він дозволяє клієнту працювати лише з інтерфейсами, не знаючи, які саме об'єкти створюються.

2. Структура шаблону «Абстрактна фабрика»



3. Класи та їх взаємодія в «Абстрактній фабриці»

AbstractFactory - оголошує інтерфейс для створення абстрактних продуктів.

ConcreteFactory - реалізує методи створення конкретних продуктів.

AbstractProduct - описує інтерфейс продуктів.

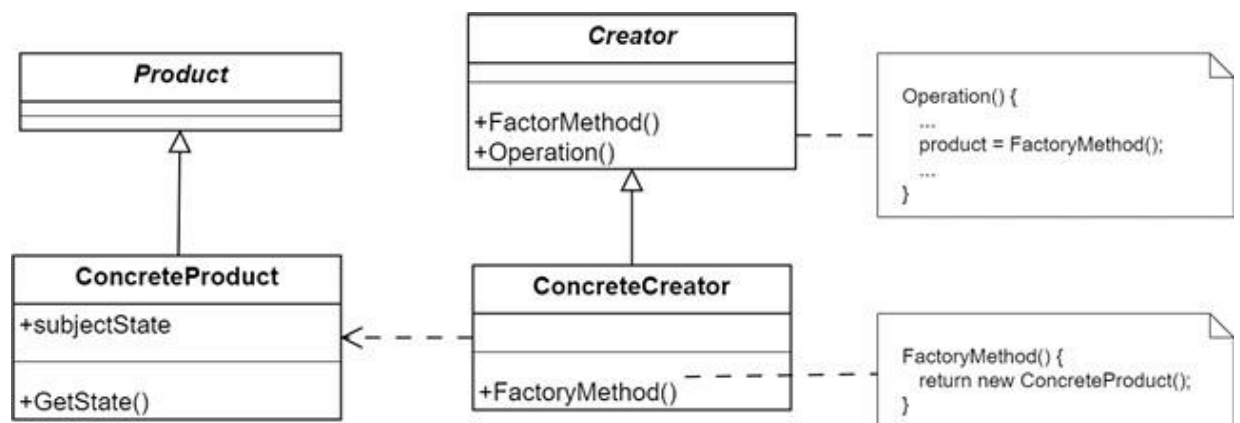
ConcreteProduct - реалізує конкретний продукт.

Client - лише з інтерфейсами **AbstractFactory** та **AbstractProduct**, не знаючи про їх реалізацію.

4. Призначення шаблону «Фабричний метод»

Шаблон «Фабричний метод» (Factory Method) визначає інтерфейс для створення об'єкта, але дозволяє підкласам вирішувати, який саме клас створювати. Він делегує створення об'єктів підкласам.

5. Структура шаблону «Фабричний метод»



6. Класи та взаємодія у «Фабричному методі»

Product - спільний інтерфейс для всіх продуктів.

ConcreteProduct - конкретна реалізація продукту.

Creator - оголошує фабричний метод

FactoryMethod().

ConcreteCreator - перевизначає FactoryMethod() для створення конкретного продукту.

Client - використовує об'єкт через базовий інтерфейс Product.

7. Відмінність між «Абстрактною фабрикою» та «Фабричним методом»

«Фабричний метод» визначає інтерфейс для створення одного типу об'єкта, але дозволяє підкласам вирішувати, який саме клас створювати. Тобто він зосереджується на створенні окремого продукту, надаючи механізм розширення без зміни існуючого коду. Основна ідея полягає в тому, щоб делегувати створення об'єкта підкласам.

«Абстрактна фабрика», своєю чергою, оперує на вищому рівні - вона створює цілі сімейства пов'язаних об'єктів, які повинні використовуватися разом. Наприклад, фабрика може створювати набір інтерфейсних елементів (кнопки, меню, поля вводу), що належать до певного стилю (Windows, macOS тощо).

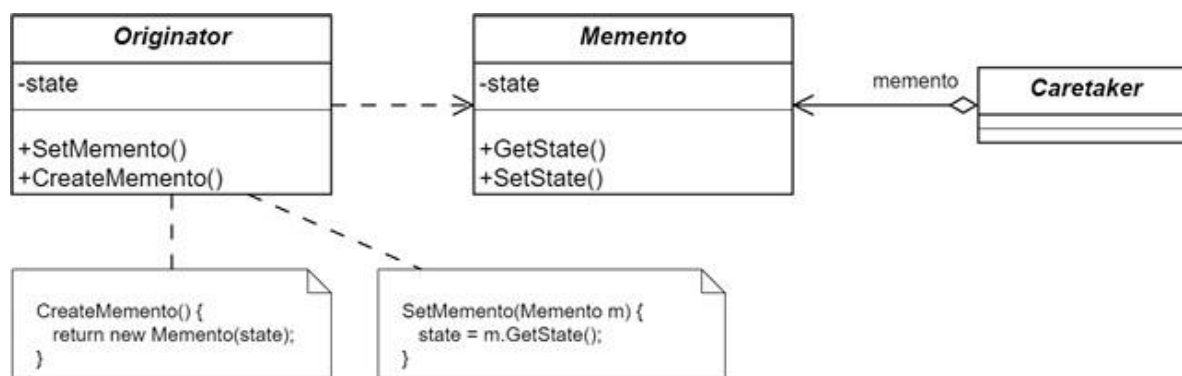
Отже, «Фабричний метод» вирішує, як створити один об'єкт, а «Абстрактна фабрика» - як створити набір сумісних об'єктів. Часто «Абстрактна фабрика» реалізується через кілька фабричних методів усередині себе.

8. Призначення шаблону «Знімок» (Memento)

Шаблон «Знімок» використовується для збереження і відновлення попереднього стану об'єкта без порушення інкапсуляції.

Він дозволяє «відкотити» об'єкт до попереднього стану (наприклад, у функції «Undo»).

9. Структура шаблону «Знімок»



10. Класи та взаємодія у «Знімку»

Originator - об'єкт, стан якого потрібно зберегти. Створює **Memento** і відновлює стан із нього.

Memento - зберігає внутрішній стан Originator.

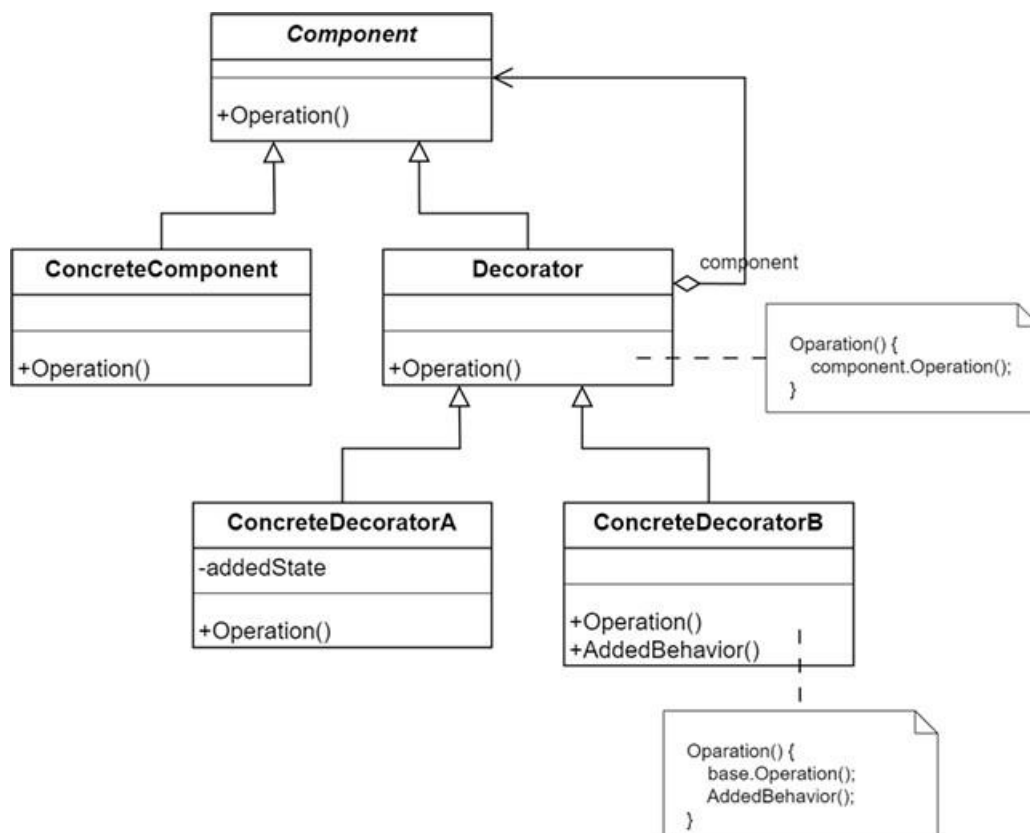
Caretaker - відповідає за збереження та відновлення Memento, але не змінює його вміст.

11. Призначення шаблону «Декоратор»

Шаблон «Декоратор» (Decorator) дозволяє динамічно додавати нову поведінку або функціональність об'єктам без зміни їх коду.

Він «обгортає» об'єкт у додатковий клас, який розширює його можливості.

12. Структура шаблону «Декоратор»



13. Класи та взаємодія у «Декораторі»

Component - базовий інтерфейс або клас.

ConcreteComponent - основний об'єкт, який може бути «обгорнутий».

Decorator - базовий клас, який реалізує інтерфейс Component і містить посилання на нього.

ConcreteDecorator - додає нову поведінку до об'єкта перед або після виклику базових методів.

14. Обмеження використання шаблону «Декоратор»

Велика кількість маленьких об'єктів ускладнює відлагодження.

Може бути важко зрозуміти послідовність викликів при вкладених декораторах.

Не підходить, коли потрібно змінювати поведінку всіх об'єктів класу одразу (краще використати наслідування)